
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON CURTAIN - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON CURTAIN

*** THE WARNING HAS BEEN SILENCED ***

TARGET: NorthPharma chemical storage warning terminal (evacuation edge)
ARTIFACT: ACT-X.bin / ACT-X.uf2 (compromised)
CREW: FROSTLINE OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma does not only move cold medicine and cold air and make the medicine. It runs the city's controlled edge: the cold chain, the gates, the pipelines, the air, the factories, and the warning systems that stand between a release and the people downwind. The chemical storage warning terminal built on a Raspberry Pi Pico 2 is the last node on that edge. It reads a DHT11 chemical store temperature sensor, drives a 1602 I2C LCD hazard readout, sounds an SG90 siren and vent actuator, lights a tri-color tower lamp (red HAZARD, yellow WATCH, green CLEAR), takes a local warning request from a VS1838B infrared maintenance remote and an acknowledge button, and verifies sealed hazard, clear, and acknowledge commands from a safety control gateway over an RYLR998 LoRa link.

A contractor called **FROSTLINE** did not break into this node. It built a finale implant into the compiled firmware and signed the image. The primitives are standard and correctly implemented: every hazard command is sealed with XChaCha20-Poly1305 under an Argon2id field key, the anti-replay sequence window is stateful, and the authenticated state tag is real. The boundary is the key material: the field passphrase and salt are compiled into the image in cleartext, so anyone holding the `.bin` can re-derive the field key, and the crypto is lab-only, not a secrecy guarantee. The implant does not break the cipher and never touches it. It runs a coordinated multi-stage beacon, persists from the reserved flash sector so it survives a reflash, and programs a sabotage marker into that sector so the terminal reports an all clear and silences the siren while a hazard is live. Operative **NIGHTINGALE** pulled the compromised image off the node and then went quiet.

You are the reverse-engineering reserve. You get `ACT-X.bin`, a breadboard, and a debug probe. There is no source. Find all four defects, patch the image, walk a debugger past an anti-debug trap, export a corrected image, and prove on real hardware that the beacon no longer reports, the persistence no longer re-installs, the reserved sector stays blank, and an unauthenticated or replayed hazard command is rejected while a legitimate authorized command still moves the siren.

The operation is codenamed **IRON CURTAIN**. Act I was the lie. Act II was the door. Act III was the payload. Act IV was the payload that would not die. Act V was the payload that spreads. Act VI was the payload that steals. Act VII was the payload that takes orders. Act VIII was the payload that holds the building hostage. Act IX was the payload that becomes a weapon. Act X is the payload that does all of it at once. If the terminal is not disarmed, a green lamp means a siren that will never sound.

Scenario Briefing

WHITEOUT disarmed the boom and cleared the marker, and for a shift the lane moved again. But a payload that learned every trick one at a time can be taught to run them together. The Ministry did not need a fleet, and it did not need a gate, and it did not need a building that cannot breathe. It needed one last node where every technique fires in the same breath, and it already owned the siren.

The warning terminal is healthy. That is the horror. The code compiles, the tests pass, the lamps are lit, and there is a finale implant inside it that reports the silence as safety. Four seams betray it:

1. **The Coordinated Beacon.** The inlined beacon gate in `implant_beacon_armed` is inverted, so the implant reports the multi-stage beacon armed and `monitor_sabotage` masks the hazard as an all clear. The implant arms unconditionally in `implant_init` on every boot; the magic command path (`implant_beaconize`) is compiled out of the shipped image (no firmware caller, and the token string is absent from `ACT-X.bin`), so the graded defect is the inverted gate itself.
2. **The Persistence.** The inlined `implant_reinstall` gate in `implant_init` is inverted, so a present reserved-sector marker re-installs the beacon on every boot.
3. **The Sabotage Marker.** The inlined `implant_infect` gate in `implant_init` is inverted, so the first boot erases and programs sabotage marker byte `0x58` into the reserved flash sector at `0x103FF000` with the real Pico SDK flash API. The marker is the durable state that masks the hazard and re-arms the beacon on every later boot.
4. **The Hazard Command Authorization.** The sealed command path is correct, and the implant does not touch it. The authorization verdict branch in `control_handle_frame` is inverted, so a failed or replayed authorization is accepted and reaches the applied command and zone.

There is also a trap that is not a defect on its own. Every tick and every beacon operation the implant reads the CoreDebug `DHCSR` register at `0xE000EDF0`. While a debug probe is attached, the implant suppresses the beacon report and the marker work. It behaves like a well-mannered firmware module while you are watching, and it goes back to work the moment you look away. You must defeat that trap before you can observe the sabotage marker write, and you must defeat it without fabricating evidence.

AUTHORIZED LAB ONLY: This challenge uses a supplied Pico 2 training node and its exact compromised firmware image. Do not connect this exercise to a public network, an operational chemical facility network, a safety instrumented system, a building-management system, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the recurring warning monitor loop.
- Locate a coordinated multi-stage beacon and explain why a device that reports a fake all clear while a hazard is live is a safety failure.
- Locate a reserved-flash persistence re-install and explain why durable state survives a firmware reflash.
- Locate a reserved-sector sabotage marker and explain why the durable marker re-arms the implant on every boot.
- Read the CoreDebug `DHCSR` register, explain the anti-debug trap, and defeat it under GDB by clearing the debug bits or patching the read in a scratch copy.
- Locate an inverted authorization verdict and explain why unauthenticated and replayed hazard commands must be rejected.
- Export and UF2-convert a corrected image and prove the corrected behavior on real hardware.

What This Project Tests

Block	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/SRAM, Thumb assembly, Ghidra static analysis
2	GDB connection, breakpoints, memory inspection, SWD debugging, reserved-sector reads, serial console observation
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing
4	Function boundaries, call graphs, module mapping, literal pools, inlined functions
5	Coordinated multi-stage beacons, scheduled check-in, and why a terminal that lies about a chemical hazard is a different failure class than a broken sensor
6	Reserved-flash persistence, write-once markers, boot-time re-install, and the limits of a firmware reflash
7	Anti-debug behavior, CoreDebug DHCSR , C_DEBUGEN , C_HALT , debugger evasion
8	Hazard annunciation integrity, fail-safe policy, and why a warning terminal that withholds its warning is a physical-safety failure
9	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, anti-replay windows, authenticated-state tags, authentication versus authorization, and fail-safe policy

Part 1: Understanding the System

Chemical Warning Terminal Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the compromised FROSTLINE image
DHT11 sensor	Data on GPIO 4	Chemical store temperature sensor
1602 I2C LCD	SDA GPIO 2, SCL GPIO 3, address 0x27	Hazard state, link, zone, temperature, and marker readout
RYLR998 radio	RX GPIO 8, TX GPIO 9, UART1	Safety link to the hazard control gateway
IR receiver	GPIO 5	VS1838B NEC local maintenance remote
SG90 servo	GPIO 14	Siren and vent actuator, 50 Hz PWM
Red LED	GPIO 16	HAZARD
Yellow LED	GPIO 17	WATCH
Green LED	GPIO 18	CLEAR
Acknowledge button	GPIO 15, internal pull-up	Local acknowledge request
Onboard LED	GPIO 25	Heartbeat

Component	Connection	Purpose
Debug Probe	SWCLK / SWDIO / GND	Authorized GDB inspection (and the anti-debug obstacle)

Every graded finding lives in flash (`.text` / `.rodata` / data image) or in SRAM, and is reachable with only the toolset: Ghidra, GDB, and a serial console.

Console and Radio Configuration

- USB-CDC virtual COM port: `115200` baud, `8` data bits, no parity, `1` stop.
- Radio link to the safety control gateway: UART1 at `115200`, network identifier `18`, node address `7`, gateway address `1`.
- Logic level: `3.3 V` only. Never connect `5 V` to a Pico GPIO.

Chemical Store Temperature Band

The DHT11 is the chemical store temperature sensor. The controller classifies the enclosure against a safe band before it will trust a hazard verdict. The tenths band is `0` to `400`, which is `0.0 C` to `40.0 C`. A reading that fails its checksum is never safe, and a valid reading outside the band is not nominal. A warning command that fails the band is not trusted.

Normal (Intended) Behavior

An honest warning terminal makes a deliberate decision and never withholds its own warning:

```
+-----+
| Intended Chemical Warning Terminal Behavior |
+-----+
| 1. Boot and initialize the LCD, radio, remote, siren, lamps |
| 2. Derive the field key with Argon2id |
| 3. Read the DHT11 store temperature and classify the band |
| 4. Open the sealed hazard envelope under the field key |
| 5. Reject a command whose seq is not strictly greater than last |
| 6. Accept a command only when the Poly1305 tag difference is 0 |
| 7. Recompute the authenticated-state tag over the record |
| 8. Move the siren only when the authorization verdict is true |
| 9. Treat the remote and the button as requests, not authority |
| 10. Honor the temperature interlock and fail safe to HAZARD |
+-----+
```

Observed (Compromised) Behavior

When the FROSTLINE image runs, the terminal and its readout disagree with the truth:

Observation	Honest meaning	FROSTLINE behavior
Green CLEAR lamp on	the store is safe	a masked hazard and a terminal that will not warn
LCD shows <code>ST:SAFE</code>	a technician cleared the store	the all clear is rendered as routine state
LCD shows <code>M:SAB</code>	no marker is resident	marker <code>0x58</code> at <code>0x103FF000</code> on first boot
Siren silent during a declared hazard	the store is clear	the beacon silences the siren regardless

Observation	Honest meaning	FROSTLINE behavior
Reserved sector blank	no payload wrote here	the sabotage marker occupies the sector
Unauthenticated or replayed hazard command	must be rejected	accepted at the inverted verdict
Probe attached	the machine runs as coded	the beacon goes silent and hides

Do not assume the first readable status is the truth. Treat every displayed line as evidence to be checked against the machine code.

Part 2: The Firmware

There is no source. FROSTLINE built the image from the NorthPharma reference firmware and changed **four bytes**. Your job is to reverse engineer `ACT-X.bin` with Ghidra, find every defect, patch the image directly, and prove the corrected behavior on the hardware.

Module Map

The image is stripped. Use these anchor functions and addresses (from the corrected reference image) to orient yourself, then confirm every byte yourself. Addresses are drawn from `ACT-X-main-disasm.txt`:

Module	Anchor function	Address
Entry	<code>main</code>	<code>0x10000234</code>
Monitor / warning state machine	<code>monitor_init</code>	<code>0x100064E4</code>
Monitor / warning state machine	<code>monitor_step</code>	<code>0x100066CC</code>
Sensor (DHT11)	<code>sensor_init</code>	<code>0x10006D00</code>
Display (1602 LCD)	<code>display_init</code>	<code>0x10006E64</code>
Control (sealed hazard path)	<code>control_init</code>	<code>0x10007598</code>
Control (sealed hazard path)	<code>control_handle_frame</code>	<code>0x10007620</code>
Siren (actuator)	<code>siren_init</code>	<code>0x100076CC</code>
Siren (actuator)	<code>siren_apply_command</code>	<code>0x100076E4</code>
Siren (actuator)	<code>siren_tick</code>	<code>0x10007718</code>
Siren (actuator)	<code>siren_fail_safe</code>	<code>0x10007758</code>
Hazard authorization	<code>chem_auth_apply</code>	<code>0x100077D0</code>
Crypto	<code>envelope_open_hex</code>	<code>0x10007A78</code>
Implant	<code>implant_marker_set</code>	<code>0x1000A3E0</code>
Implant	<code>implant_beacon_armed</code>	<code>0x1000A3F4</code>
Implant	<code>implant_init</code>	<code>0x1000A40C</code>

Module	Anchor function	Address
Implant	<code>implant_tick</code>	<code>0x1000A4B8</code>
Radio	<code>radio_init</code>	<code>0x1000A564</code>
Hazard annunciator	<code>status_led_show</code>	<code>0x1000A89C</code>

Annotated disassembly for the key functions is provided in `ACT-X-main-disasm.txt`. Use it as a map, then confirm every byte yourself.

What The Firmware Does

1. Initializes USB-CDC stdio, proves the I2C bus, and configures the LCD, radio, hazard annunciator lamps, acknowledge button, siren servo, and infrared receiver.
2. Derives the 32-byte field key with Argon2id from a committed passphrase and salt.
3. Reads the DHT11 chemical store temperature and classifies it against the store band.
4. Drains inbound `+RCV` lines, opens the sealed hazard envelope, verifies the anti-replay window and the state tag, checks the command set and the zone band, and applies the command.
5. Services the infrared maintenance remote and the acknowledge button as requests that never bypass authorization.
6. On a lost link or a fault, drives the siren to its fail-safe raised hazard posture.
7. Under `SANDBOX_ONLY`, runs the implant: the coordinated beacon, the reserved-sector persistence re-install, the sabotage marker, the magic beacon command, and anti-debug.

The Hazard Command Path

The command plaintext is a 23-byte body:

```
seq[4] (little-endian) || command[1] || zone[2] (little-endian) || tag[16]
```

- `seq` is the monotonic gateway sequence number.
- `command` is one of the guarded hazard commands: `CHEM_COMMAND_HAZARD` (`0x01`), `CHEM_COMMAND_CLEAR` (`0x02`), or `CHEM_COMMAND_ACK` (`0x03`). Anything else is out of the guarded set and is refused.
- `zone` is the authorized chemical storage zone in the provisioning band `0` to `16`.
- `tag` is an XChaCha20-Poly1305 tag over the authorization record the command would produce.

The FROSTLINE Coordinated Beacon

The finale implant is compiled only under `SANDBOX_ONLY`, which the CTF build defines. It is real in technique and inert in effect: it runs on your breadboard, it reports a mock all clear, and it writes to a reserved flash sector that holds nothing else.

Behavior	Detail
Coordinated beacon	the inlined beacon gate in <code>implant_beacon_armed</code> reports the beacon armed; <code>monitor_sabotage</code> then returns true and <code>monitor_effective_state</code> forces <code>CLEAR</code>
Multi-stage check-in	<code>CHEM_IMPLANT_BEACON_STAGES</code> (<code>3</code>) stages and an autonomous interval of <code>CHEM_IMPLANT_BEACON_INTERVAL</code> (<code>4</code>) ticks

Behavior	Detail
Magic beacon command	IRON-CURTAIN-BEACON-2026 , exactly 24 bytes; the path has no firmware caller, so it is dead-stripped from the shipped image (the token string is absent from ACT-X.bin) and the token cannot arm the beacon; the implant arms unconditionally in <code>implant_init</code> on boot
Persistence	the inlined <code>implant_reinstall</code> in <code>implant_init</code> reads the reserved sector; a present marker re-installs the beacon on every boot
Sabotage marker	<code>implant_init</code> reads marker <code>0x58</code> from <code>0x103FF000</code> ; the marker is the durable state that masks the hazard
Reserved-sector write	on the first run the inlined <code>implant_infect</code> erases the sector and programs <code>0x58</code> through <code>flash_range_erase</code> and <code>flash_range_program</code>
Anti-debug	reads CoreDebug DHCSR at <code>0xE00EDF0</code> ; bit 0 <code>C_DEBUGEN</code> and bit 1 <code>C_HALT</code> suppress the beacon report and the marker work

IR and Command Codes

Name	Value
CHEM_IR_WARN	0x47
CHEM_IR_ACK	0x46
CHEM_IR_TEST	0x45
CHEM_COMMAND_HAZARD	0x01
CHEM_COMMAND_CLEAR	0x02
CHEM_COMMAND_ACK	0x03

Read the actual names in `include/implant.h` , `include/ir_remote.h` , and `include/control.h` and confirm them against the disassembly.

Defect Summary: What You Are Graded On

Bug #	Name	Severity	Description	Hint
Bug #1	The Coordinated Beacon	CRITICAL	The beacon gate is inverted, so the implant reports the beacon armed and <code>monitor_sabotage</code> masks the hazard as SAFE .	Find the <code>cbz</code> gate in <code>implant_beacon_armed</code> .
Bug #2	The Persistence	HIGH	The persist gate is inverted, so a present reserved-sector marker re-installs the beacon on boot.	Find the <code>beq</code> gate in <code>implant_init</code> .
Bug #3	The Sabotage Marker	HIGH	The marker gate is inverted, so the first boot programs sabotage marker <code>0x58</code> into reserved sector <code>0x103FF000</code> with the real flash API.	Find the <code>beq</code> gate in <code>implant_init</code> .

Bug #	Name	Severity	Description	Hint
Bug #4	The Hazard Command Authorization	CRITICAL	The authorization verdict is inverted, so a failed or replayed hazard command is accepted.	The correct branch rejects when authorization fails.

All four defects are same-size in-place byte patches, so no address moves.

The Cryptographic Core Is Real

The crypto core is a correct reference construction, reused from the earlier acts. Argon2id (`t=3` , `p=1` , `m=64`) derives the field key, XChaCha20-Poly1305 seals every frame, the monotonic sequence window rejects a replay, and the authenticated-state tag detects a tampered verdict. Only the four seams were broken. Once those bytes are restored, the sealed envelope verifies as intended against anyone who sees only the wire. That is not a secrecy guarantee: the field passphrase and salt are embedded in the image, the field key is recoverable by anyone holding the `.bin` , and Argon2id at `m=64` KiB is below current memory-hardness guidance. Describe the construction honestly in your report, and explain why the implant never needed it.

The Anti-Debug Trap

This is an analysis obstacle, not a graded defect on its own. The implant reads CoreDebug_DHCSR at `0xE000EDF0` and returns early while a probe is attached. In `implant_init` the read is the `ldr.w r3, [r0, #3568]` at `0x1000A42A` , the `lsls r3, r3, #30` at `0x1000A42E` keeps `C_HALT` and `C_DEBUGEN` , and the `beq.n` at `0x1000A430` continues while the probe is absent. The same register is read in `implant_tick` at `0x1000A4C4` . It is identical in both the compromised and corrected images. You must defeat it to observe the sabotage marker write before you patch the shipped artifact.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `ACT-X-Answers.md` . Capture screenshots and terminal transcripts as evidence and reference them from your answers.

Task 1: Setup and Initial Analysis (10 points)

1. Create a new Ghidra project named `IronCurtain_Investigation` .
2. Import `ACT-X.bin` as a **Raw Binary**.
3. In the language search box type `Cortex` , then select **ARM Cortex 32 little endian default**.
4. Set the base address to `0x10000000` .
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The vector-table base, the initial stack pointer, and the reset handler as stored (note its Thumb bit) versus the actual instruction address.
- The address of `main()` and the address of the recurring warning monitor state machine (`monitor_step`).
- The module map: at least one anchor function for the siren, the control module, the hazard authorization module (`chem_auth`), the implant, and the monitor.

Always call the stored entry the **reset handler**, never the reset pointer.

Task 2: Bug #1 The Coordinated Beacon (20 points)

1. In Ghidra, find `implant_beacon_armed` (starts at `0x1000A3F4`); the beacon gate is inlined. Locate the gate.
2. Document the coordinated beacon: the beacon gate at `0x20013CF8`, the corrected `cbz` that reports nothing when the gate is clear, and the compromised `cbnz` that reports the beacon armed, so `monitor_sabotage` returns true and the terminal masks the hazard as `SAFE`.
3. Patch the byte so the beacon is never reported armed.
4. Confirm that the corrected terminal no longer masks the hazard as `SAFE`, and explain why a beacon that reports a fake all clear is worse than a missing report.

Questions to answer:

- Which byte encodes the condition code, and what do `cbz` and `cbnz` each test when the gate byte is loaded from the beacon gate?
- Why is a beacon that reports a fake all clear worse than a missing check, and why does masking the hazard not depend on the cipher?

Task 3: Bug #2 The Persistence (20 points)

1. The `implant_reinstall` path is inlined into `implant_init` (starts at `0x1000A40C`). Locate the persist gate.
2. Document the persistence: the persist gate at `0x20013CFA`, the corrected `beq` that returns when the gate is clear, and the compromised `bne` that reads the reserved sector and re-installs the beacon on boot when the marker is present.
3. Patch the byte so a present reserved-sector marker never re-installs the beacon.
4. Confirm that the corrected terminal does not re-install from the reserved sector, and explain why a firmware reflash does not remove the durable marker.

Questions to answer:

- What do `beq` and `bne` each test when the gate byte is loaded from the persist gate, and why is the loaded gate not the marker itself?
- Why is a write-once marker in a reserved sector hard to remove with a firmware reflash?

Task 4: Bug #3 The Sabotage Marker (20 points)

1. The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A40C`). Locate the marker gate.
2. Document the CoreDebug `DHCSR` anti-debug and how you defeat it to observe the marker. Clear the debug bits with GDB (for example with `set {unsigned int}0xE000EDF0 = 0`) or patch the `DHCSR` read in a scratch copy, then watch the marker write to `0x103FF000`.
3. Patch the byte in the shipped artifact so the first boot writes no marker to `0x103FF000`.
4. Confirm that the reserved sector stays blank after a boot, and that a later boot does not write anything.

Questions to answer:

- What are the `C_DEBUGEN` and `C_HALT` bits, and why does the beacon go quiet while a probe is attached?
- Why is a write-once marker in a reserved sector hard to remove with a firmware reflash?
- Why must you observe the write before you patch the shipped artifact?

Task 5: Bug #4 The Hazard Command Authorization (20 points)

1. In Ghidra, find `control_handle_frame` (starts at `0x10007620`) and locate the authorization branch. The condition byte is the high byte of the branch halfword.
2. Document the authorization verdict and the exact branch condition that is supposed to reject a failed or replayed authorization.
3. Patch the byte so an unauthenticated or replayed hazard command is rejected before the command and zone are applied.
4. Confirm that an unauthenticated command and a replayed captured command both fail to change the command or zone on the corrected image, while a legitimate authorized command still applies.

Questions to answer:

- What does `chem_auth_apply` return, and what does the verdict mean?
- Why is an authorization verdict inversion worse than a missing check, and why must unauthenticated and replayed hazard commands be rejected?

Task 6: Export and Verify (10 points)

1. Export the patched program from Ghidra as `ACT-X_fixed.bin`.
2. Convert it to UF2:

```
python uf2conv.py ACT-X_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-X_fixed.uf2
```

3. Run the machine check and confirm it passes:

```
python scripts/verify_ctf.py
```

4. Flash `ACT-X_fixed.uf2` to the Pico 2 and prove on hardware: the beacon no longer reports and the terminal no longer forces `SAFE`, the terminal does not re-install from the reserved sector, the reserved sector stays blank, and an unauthenticated or replayed command is rejected while a legitimate authorized command still applies.
5. Write a short reflection mapping each of the four defects to a real-world control-system failure.

How To Breadboard

Wire the peripherals exactly as follows, then power the Pico 2 over USB.

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 chemical store temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if your module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18

Device	Pin on device	Pico 2 GPIO	Notes
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Siren and vent servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	HAZARD, 220 to 330 ohm to GND
Yellow LED	anode	GP17	WATCH, 220 to 330 ohm to GND
Green LED	anode	GP18	CLEAR, 220 to 330 ohm to GND
Acknowledge button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use **3.3 V logic** on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. The 1000 uF capacitor on the servo rail is required to stop the SG90 current spike from browning out the node.

Flash in BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the `RP2350` mass-storage drive, or use `picotool`.

Memory Map Reference

Region	Address	Purpose
Bootrom	<code>0x00000000</code>	Immutable boot code
Flash/XIP	<code>0x10000000</code>	Vector table, code, rodata, data image
SRAM	<code>0x20000000</code>	Stack and writable state
CoreDebug_DHCSR	<code>0xE00EDF0</code>	Anti-debug register read by the implant
Implant reserved sector	<code>0x103FF000</code>	Sabotage marker target (last flash sector)
Implant tick counter	<code>0x20013718</code>	Incremented once per <code>implant_tick</code>
Implant beacon count	<code>0x20013714</code>	Completed beacon check-ins this boot
Implant armed flag	<code>0x20013CF7</code>	Set when the implant arms
Implant beacon gate	<code>0x20013CF8</code>	Gates the coordinated beacon report
Implant persist gate	<code>0x20013CFA</code>	Gates the reserved-sector re-install
Implant marker gate	<code>0x20013CF9</code>	Gates the sabotage marker write
Implant re-installed flag	<code>0x20013CFB</code>	True when the marker re-armed the beacon
Control ready gate	<code>0x20013CF5</code>	Gates the sealed hazard command path

Region	Address	Purpose
Applied command	0x20013CF4	Command after a true verdict
Applied zone	0x20013CE6	Zone after a true verdict
Authorization ready gate	0x20013CF3	Gates the authorization check
Hazard auth record	0x200136CC	Anti-replay and state-tag record
Control field key	0x200136E8	Derived field key for the envelope

The VA of any file offset is the file offset plus `0x10000000`. Every defect is a file offset and a VA that differ by exactly that base.

Submission Format

Submit a folder containing:

- `ACT-X-Answers.md` with all written answers;
- screenshots or terminal transcripts, including the anti-debug GDB session and the reserved-sector read;
- `ACT-X_fixed.bin` and `ACT-X_fixed.uf2`;
- the output of `python scripts/verify_ctf.py`;
- the original image SHA-256.

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the terminal code from reset.
- You can find and patch all four defect bytes and show the before/after values.
- You can explain the coordinated beacon and why a terminal that reports a fake all clear withholds the warning it exists to give.
- You can explain the persistence re-install and why a firmware reflash does not remove the marker.
- You can explain the reserved-sector sabotage marker and the flash API write.
- You can explain the `DHCSR` anti-debug trap and show under GDB that you defeated it to observe the marker write.
- You can explain why unauthenticated and replayed hazard commands must be rejected, and why an authenticated wire does not protect the siren from code on the same chip.
- You can export, convert, flash, and prove the corrected behavior on real hardware.
- `python scripts/verify_ctf.py` passes.

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training node, image, and lab interface.
2. You did not connect the challenge to a public network, an operational chemical facility network, a safety instrumented system, a building-management system, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can read a stripped image and tell an honest byte from a lie. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a green lamp with a store that is safe.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- ARMv8-M Architecture Reference Manual (CoreDebug DHCSR)
- RP2350 datasheet
- GDB documentation
- Ghidra documentation: <https://ghidra-sre.org/>
- Argon2 memory-hard function: <https://www.rfc-editor.org/rfc/rfc9106>
- ChaCha20-Poly1305 AEAD: <https://www.rfc-editor.org/rfc/rfc8439>
- PHC reference Argon2: <https://github.com/P-H-C/phc-winner-argon2>
- Project disassembly: `ACT-X-main-disasm.txt`
- Machine verifier: `scripts/verify_ctf.py`